

FACULDADE DE TECNOLOGIA - FATEC

Curso: [CURSO/SEMESTRE - INSERIR]

Disciplina: Projeto Atribuído - Integração Front-End / Back-End / Banco de Dados

BAB-RANK: Evolução do Projeto SteamTwo com Integração PostgreSQL, Docker Compose e APIs Externas

Autor: Pedro Emanuel da Silva de Oliveira

RA: _____ [INSERIR RA]

Orientador: [INSERIR NOME DO PROFESSOR]

Cidade — 2026

BAB-RANK

Evolução do Projeto SteamTwo com Integração PostgreSQL, Docker
Compose e APIs Externas

Pedro Emanuel da Silva de Oliveira

RA: _____

Trabalho apresentado como requisito da disciplina Projeto Atribuído

FOLHA DE ROSTO

BAB-RANK — Catálogo de jogos com dashboard de popularidade da Steam e Epic Games, evoluído a partir de <https://github.com/DavidSilvaProg/steamtwo>

- **Novo repositório:** <https://github.com/<SEU-USUARIO>/bab-rank> [INSERIR LINK FINAL]
- **Pasta local:** /home/pedro/dev/Fatec/bab-rank
- **Integrante:** Pedro Emanuel da Silva de Oliveira — RA: _____

Trabalho desenvolvido individualmente conforme rubrica (10 pontos).
Pasta renomeada de bab-steam-epic para bab-rank em 31/08/2026.

SUMÁRIO

1. Introdução
 2. Referencial Teórico — SteamTwo
 3. Metodologia e Integração Back-End / Banco de Dados
 4. Melhorias Implementadas (6 itens documentados)
 5. Instruções de Execução
 6. Testes e Resultados
 7. Evidências de Funcionamento (Prints)
 8. Conclusão
 9. Referências
-

1. INTRODUÇÃO

O projeto BAB-RANK tem como objetivo evoluir o projeto-base SteamTwo, consolidando a integração entre front-end (React/Vite), back-end (Node.js/Express) e banco de dados (PostgreSQL 17), conforme exigências da rubrica:

- 3,0 pts — back-end funcionando e integrado ao PostgreSQL
- 3,0 pts — melhorias relevantes autorais
- 2,0 pts — integração, testes e qualidade técnica
- 2,0 pts — PDF completo conforme ABNT

O projeto foi desenvolvido solo, com foco prioritário na integração via Docker Compose e população via APIs externas (IGDB, Steam, Epic). O nome foi alterado de steamtwo para bab-rank e a logo do site foi refeita (BAB RANK com coroa).

2. REFERENCIAL — STEAMTWO

Funcionalidades originais:

- Dashboard com mais jogados agora, média da última semana, popularidade histórica e recorde monitorado
- Catálogo pesquisável e filtrável por loja e gênero
- Ranking combinado transparente: cada posição normalizada por $100 \times (N - \text{posição} + 1) / N$, índice combinado é média das fontes
- Coleta Steam, Epic Games e IGDB com snapshots imutáveis
- Fallback visual quando PostgreSQL não configurado

Stack: React 19 + Vite 6 (front em src/), Express 5 + pg 8 (back em server/), node-pg-migrate (migrations/001_create_steamtwo_domain.js), vitest + supertest.

Problema original: banco vazio sem seed; IGDB popularity removido da API (400), rankings com ranks duplicados e sem criação de jogos.

3. INTEGRAÇÃO BACK-END COM BANCO DE DADOS

3.1 Arquitetura

```
[Browser] -> [Vite/React em src/ | dist/client]
            -> [Express server/index.js:14 em :3001 -> BAB-RANK API]
            -> [createPool(DATABASE_URL) server/db/pool.js:6]
            -> [PostgreSQL 17 - postgres:17-alpine]
            -> [Tabelas: games, genres, game_genres, store_listings,
sync_runs, ranking_snapshots, ranking_entries, game_rankings]
```

- server/config.js:6 lê DATABASE_URL do .env
- server/index.js:14-18 cria pool e repository; se DATABASE_URL ausente, healthCheck retorna {status:"not-configured", mode:"demo"} (fallback)

- Com DB, healthCheck executa `SELECT 1` e `/api/health` retorna `{"status":"ok","database":{"status":"connected"}}` (validado em `curl http://127.0.0.1:3001/api/health`)
- **Correção 1 — `server/db/job-repository.js:118`:**
`jsonb_build_object('records',$4::int)` — PG17 exigia cast, erro `could not determine data type of parameter $4`
- **Correção 2 — `server/db/job-repository.js:50`:** distinção Pool vs Client via `totalCount` — erro `Client has already been connected`
- **Correção 3 — `server/db/catalog-read-repository.js:32`:**
adicionado `g.updated_at AS "updatedAt"` e `mapGame` com `updatedAt`, `GROUP BY` inclui `g.updated_at`
- **Correção 4 — `server/services/catalog-service.js:16`:** `rankingView` usa `game.updatedAt` ?? `mockUpdatedAt` e `dashboard` usa `latestUpdate` real do banco, não hardcoded 2026-08-24

3.2 Migrações

- `migrations/001_create_steamtwo_domain.js` cria 9 tabelas + 4 enums (`store_name`, `sync_state`, `snapshot_status`, `ranking_period`) + função `steamtwo_prevent_snapshot_mutation()` (snapshots imutáveis)
- Comandos: `npm run db:migrate (up)` e `npm run db:rollback (down)` — testado: `MIGRATION 001_create_steamtwo_domain (UP)` com `CREATE TABLE` e `Migrations complete!`
- Evidência atual: `SELECT name, run_on FROM pgmigrations;` retorna `001_create_steamtwo_domain | 2026-08-31, \dt` lista 9 relações, `SELECT count(*) FROM games = 239`

3.3 Docker Compose

Antes (original): apenas postgres em `docker-compose.yml:1` com `5432:5432`

Depois (BAB-RANK):

`services:`

```
postgres: # postgres:17-alpine, 5433:5432 (host 5433 para
evitar conflito com postgres-db em 5432), healthcheck
pg_isready, volume external bab-steam-epic_steamtwo_pgdata
api: # build Dockerfile (node:20-alpine, npm ci + npm run
build), 3001:3001, DATABASE_URL=postgres://
steamtwo:steamtwo@postgres:5432/steamtwo (interno),
depends_on postgres healthy, healthcheck wget /api/health
migrate: # mesmo build, command npm run db:migrate, depende de
postgres healthy
```

`volumes:`

```
steamtwo_pgdata:
  external: true
  name: bab-steam-epic-steamtwo_pgdata # preserva dados após
  rename bab-steam-epic -> bab-rank
```

- Dockerfile multi-stage: FROM node:20-alpine, npm ci, COPY ., RUN npm run build (gera dist/client/index.html, dist/server/index.js), EXPOSE 3001, HEALTHCHECK wget /api/health, CMD ["node", "server/index.js"]
- Host DATABASE_URL=postgres://steamtwo:steamtwo@localhost:5433/steamtwo (externo), container DATABASE_URL=postgres://steamtwo:steamtwo@postgres:5432/steamtwo (interno)
- Validação: docker compose ps mostra bab-rank-postgres-1 healthy e bab-rank-api-1 healthy, docker logs bab-rank-api-1 mostra BAB-RANK API disponível na porta 3001

3.4 População via API Externa

Credenciais IGDB em .env:4-5 com

TWITCH_CLIENT_ID=m75esjfuhxjb8cy9vv5e27hbzgif7p e

TWITCH_CLIENT_SECRET=nni8jae31shtxon5enciwchay77637 (não commitado, .gitignore:4).

- **Fix IGDB:** server/integrations/igdb-client.js:44 trocou popularity (400 Invalid field) por rating, aggregated_rating, total_rating, follows, hypes e filtro external_games.category=(1,26) para garantir store_listings; normalizers.js:22 fallback para total_rating
- **Syncs executados:**
 - npm run sync:catalog → 100 jogos (IGDB total_rating desc)
 - npm run sync:popularity → 100
 - npm run sync:rankings → 139 (steam 119 + epic 20) com reindex rank=index+1 (server/jobs/rankings.js:28) para evitar duplicate key ranking_entries_uniq_snapshot_id_position e auto-criação de games faltantes (job-repository.js:142)
- **DB atual (31/08):** games=239, store_listings=239, genres=23, ranking_snapshots=2, ranking_entries=139, sync_runs=14

4. MELHORIAS IMPLEMENTADAS

#	Melhoria	Arquivo:linha	Descrição autoral	Evidência
1				

#	Melhoria	Arquivo:linha	Descrição autoral	Evidência
	Docker Compose completo	docker-compose.yml:1, Dockerfile:1	Evoluiu compose de 1 para 3 serviços (postgres+api+migrate), volume external para preservar após rename, docker compose up --build reproduzível	docker compose ps healthy, bab-rank-api:1
2	Mapeamento porta 5433 + rename	docker-compose.yml:9, .env:3, package.json:2	Evita conflito postgres-db 5432, pasta bab-rank, package.json name bab-rank, index.html title BAB-RANK	docker ps 0.0.0.0:5433->5432
3	Logo BAB-RANK	src/App.jsx:66, src/styles.css:12	Brand BAB RANK com <Crown weight=fill> azul, display:inline-flex, footer BAB-RANK	Print / header
4	IGDB fix + população externa	igdb-client.js:44, normalizers.js:22, job-repository.js:142	Corrigiu 400 popularity, filtro Steam/Epic, auto-cria jogos em rankings	sync:catalog 100, games=239
5	Top 100 Rankings	src/App.jsx:102	Reescrito Rankings para GET /api/rankings? period=&limit=100 com 3 botões (Agora/Semana/De sempre), mostra 100 linhas (antes só 5)	curl /api/rankings? limit=100 → 100
6	Filtro quadro “Última Semana”	src/App.jsx:74	Home: store só afeta quadro week via GET /api/rankings? period=week&store=...&limit=5, hero/topFive permanecem all (pedido: não mudar tela toda)	Print filtro Steam/Epic no quadro
7	Banner “Ver mais” 3 linhas	src/App.jsx:68, src/styles.css:32	Hero com useState expanded, .hero-summary.clamped { -webkit-line-clamp:3 }, botão Ver mais/Ver menos	Print banner expandido
8	Data correta	catalog-read-repository.js:32, catalog-service.js:16, src/App.jsx:142	Backend retorna updatedAt real do banco (g.updated_at), frontend Detail usa formatDate(game.updatedAt) em vez de hardcoded 24 de ago. 2026	curl /api/games/dcs-world-a-10c-warthog → 2026-08-31, detail última

#	Melhoria	Arquivo:linha	Descrição autoral	Evidência
				atualização: 31 de agosto de 2026

5. INSTRUÇÕES PARA EXECUTAR O PROJETO

Requisitos: Docker 29+, Node 20+ (via fnm), Git

```
# 1. Clonar (ou usar pasta local)
git clone https://github.com/<SEU-USUARIO>/bab-rank.git
cd bab-rank

# 2. Instalar
fnm exec --using=lts-latest npm install

# 3. Configurar env (não commitar .env)
cp .env.example .env
# Editar: DATABASE_URL=postgres://
      steamtwo:steamtwo@localhost:5433/steamtwo
#       TWITCH_CLIENT_ID=m75esjfuhrjb8cy9vv5e27hbzgif7p
#       TWITCH_CLIENT_SECRET=nni8jae31shtxon5enciwchay77637

# 4. Subir banco + api via Docker Compose (recomendado)
docker compose up -d --build
# ou apenas postgres para dev local:
# docker compose up -d postgres

# 5. Migrações
fnm exec --using=lts-latest npm run db:migrate
# Reverter se necessário: fnm exec --using=lts-latest npm run
      db:rollback

# 6. Popular via API externa
fnm exec --using=lts-latest npm run sync:catalog      # 100 IGDB
fnm exec --using=lts-latest npm run sync:popularity  # 100
fnm exec --using=lts-latest npm run sync:rankings    # 139 Steam/
      Epic

# 7. Validar integração
curl http://127.0.0.1:3001/api/health
# Esperado: {"status":"ok","database":{"status":"connected"}}
```

8. Dev local (sem Docker api)

fnm exec --using=lts-latest npm run dev:api # :3001

fnm exec --using=lts-latest npm run dev # :5173

9. Produção via Docker (api serve front)

Após build, acessar http://127.0.0.1:3001/ e /jogos/dcs-world-a-10c-warthog

Frontend: http://127.0.0.1:3001/ (Docker) ou http://127.0.0.1:5173/ (Vite dev)

API: http://127.0.0.1:3001/api/health, /api/games, /api/rankings?limit=100, /api/dashboard?store=steam|epic

6. TESTES REALIZADOS E RESULTADOS

Teste	Comando	Resultado esperado	Resultado observado (31/08)
Migração UP	npm run db:migrate	Migrations complete! + 9 tabelas	OK — CREATE TABLE games... e Migrations complete!
Migração DOWN/UP	db:rollback + db:migrate	Reversível	OK — DROP TABLE e recriação
pgmigrations	psql -c "SELECT name FROM pgmigrations"	1 linha	OK — 001_create_steamtwo_domain
	psql -c "\dt"	9 tabelas	OK — games, genres, game_genres, store_listings, sync_runs, ranking_snapshots, ranking_entries, game_rankings, pgmigrations
Counts	psql -c "SELECT count(*) FROM games"	>0	OK — 239, store_listings 239, ranking_snapshots 2, ranking_entries 139
/api/health	curl :3001/api/health	connected	OK — {"status":"ok","database":{"status":"connected"}}
/api/games			

Teste	Comando	Resultado esperado	Resultado observado (31/08)
/api/games/:slug	curl :3001/api/games?limit=3	200 + pagination	OK — items[0].slug dcs-world-a-10c-warthog, updatedAt 2026-08-31
	curl :3001/api/games/dcs-world-a-10c-warthog	200 + updatedAt	OK — updatedAt 2026-08-31T18:16:14.028Z (antes 2026-08-24 mock)
/api/rankings limit=100	curl :3001/api/rankings?limit=100	100	OK — len 100
/api/dashboard store	curl :3001/api/dashboard?store=steam	topFive steam	OK — hero DCS World steam, epic → Disciples II
/ (front) npm test	curl :3001/	HTML	OK — <title>BAB-RANK
	npm test	23 passed	OK — 5 passed, 23 passed
npm build	npm run build	dist/client/index.html	OK — vite 6.4.2 built 483KB
test:sites	npm run test:sites	4 passed	OK — pass 4, fail 0
Docker health	docker compose ps	healthy	OK — bab-rank-postgres-1 healthy, bab-rank-api-1 healthy
Banner	clique Ver mais	expande	OK — hero-summary clamped 3 linhas → expanded
Detail data	/jogos/dcs...	data correta	OK — Última atualização: 31 de agosto de 2026

7. EVIDÊNCIAS DE FUNCIONAMENTO

- docker compose ps — bab-rank-postgres-1 0.0.0.0:5433->5432 healthy, bab-rank-api-1 0.0.0.0:3001->3001 healthy
- docker logs bab-rank-api-1 — BAB-RANK API disponível na porta 3001

3. `curl /api/health — {"status":"ok","database":{"status":"connected"}}`
4. `psql \dt e pgmigrations + SELECT count(*) FROM games → 239`
5. `curl /api/games/dcs-world-a-10c-warthog — updatedAt 2026-08-31`
6. `npm test — 23 passed; npm run build — 483KB; test:sites — 4 passed`
7. Frontend `http://127.0.0.1:3001/` — header BAB RANK com coroa, banner com Ver mais (3 linhas), dashboard com filtro quadro só, top strip
8. `http://127.0.0.1:3001/rankings` — 100 linhas com filtros Agora/Semana/De sempre
9. `http://127.0.0.1:3001/jogos/dcs-world-a-10c-warthog` — data 31/08/2026

Para gerar via terminal:

```
docker compose ps > docs/evidencia-ps.txt
curl -s http://127.0.0.1:3001/api/health | jq . > docs/evidencia-health.json
curl -s http://127.0.0.1:3001/api/games/dcs-world-a-10c-warthog | jq .updatedAt > docs/evidencia-updatedAt.txt
docker exec bab-rank-postgres-1 psql -U steamtwo -d steamtwo -c "\dt" > docs/evidencia-dt.txt
docker exec bab-rank-postgres-1 psql -U steamtwo -d steamtwo -c "SELECT count(*) FROM games;" > docs/evidencia-count.txt
```

8. CONCLUSÃO

A integração foi comprovada: Docker Compose orquestra Postgres 17 + API Node (com fixes de Pool e cast), migrações executam e `api/health` retorna `connected`. O sistema serve front e API na mesma porta em produção. A população via IGDB/Steam/Epic foi corrigida (popularity → total_rating, filtro Steam/Epic, auto-criação) e o banco contém 239 jogos com rankings. Melhorias de usabilidade (logo BAB-RANK, top 100, filtro quadro, banner Ver mais, data correta) atendem à rubrica de 3,0 pts.

9. REFERÊNCIAS

- DavidSilvaProg/steamtwo — <https://github.com/DavidSilvaProg/steamtwo>
- Documentação PostgreSQL 17, Docker, Node.js 20, Express 5, Vite 6, IGDB API, Twitch OAuth

ANEXO: Link do repositório: <https://github.com/<SEU-USUARIO>/bab-rank>

Autor: Pedro Emanuel da Silva de Oliveira — RA: _____ (inserir)

Data: 31 de Agosto de 2026